Indian Institute of Technology Jodhpur

# CSL 7640: Natural Language Understanding

## Assignment-2


**Name:** Laksh Mendpara
**Roll Number:** B23CS1037
**Course:** CSL 7640: Natural Language Understanding


March 25, 2026

# Contents


**1 Problem 1: Learning Word Embeddings From IIT Jodhpur Data** **2**

1.1 Task 1: Dataset Preparation . . . . . 2

1.2 Task 2: Model Training . . . . . 3

1.3 Task 3: Semantic Analysis . . . . . 4

1.3.1 Nearest Neighbors . . . . . 5

1.3.2 Analogy Experiments . . . . . 5

1.4 Task 4: Visualization . . . . . 6

1.5 Deliverables for Problem 1 . . . . . 7

**2 Problem 2: Character-Level Name Generation Using RNN Variants** **8**

2.1 Task 0: Dataset . . . . . 8

2.2 Task 1: Model Implementation . . . . . 8

2.3 Task 2: Quantitative Evaluation . . . . . 9

2.4 Task 3: Qualitative Analysis . . . . . 10

2.5 Deliverables for Problem 2 . . . . . 10

**3 Submission File Checklist** **11**

**4 Conclusion** **11**

# 1 Problem 1: Learning Word Embeddings From IIT Jodhpur Data

## 1.1 Task 1: Dataset Preparation

For Problem 1, I built the corpus from multiple IIT Jodhpur sources, including the mandatory academic regulation pages, department and academic program pages, faculty profile pages, research pages, student-life pages, and course-related content. The collection pipeline first crawled focused English pages from the IIT Jodhpur domain and then filtered noisy pages such as announcements, recruitment pages, newsletters, and very short low-signal pages.

The preprocessing pipeline included the following steps:

- removal of boilerplate text, navigation fragments, repeated page sections, URLs, emails, and formatting artifacts,
- English-only filtering,
- tokenization,
- lowercasing,
- removal of excessive punctuation and non-textual content,
- phrase normalization for common academic expressions such as `comprehensive_exam`, `faculty_member`, and `mtech_phd`,
- segment-level deduplication before building the final training corpus.

The cleaned corpus file used for Word2Vec training is `q1/output/processed_corpus.txt`.

Table 1: Problem 1 dataset statistics

| Statistic | Value |
|---|---:|
| Raw documents collected | 181 |
| Retained documents after filtering | 177 |
| Training segments | 1545 |
| Duplicate segments skipped | 52 |
| Total tokens | 54,382 |
| Vocabulary size | 6,950 |
| Average segment length | 35.2 tokens |
| Median segment length | 35.0 tokens |
| Learned academic bigrams | 40 |

The ten most frequent tokens in the processed corpus were `program`, `research`, `student`, `technology`, `science`, `application`, `course`, `use`, `physics`, and `area`. This distribution is consistent with the target domains of academic regulations, research content, and student-facing information.

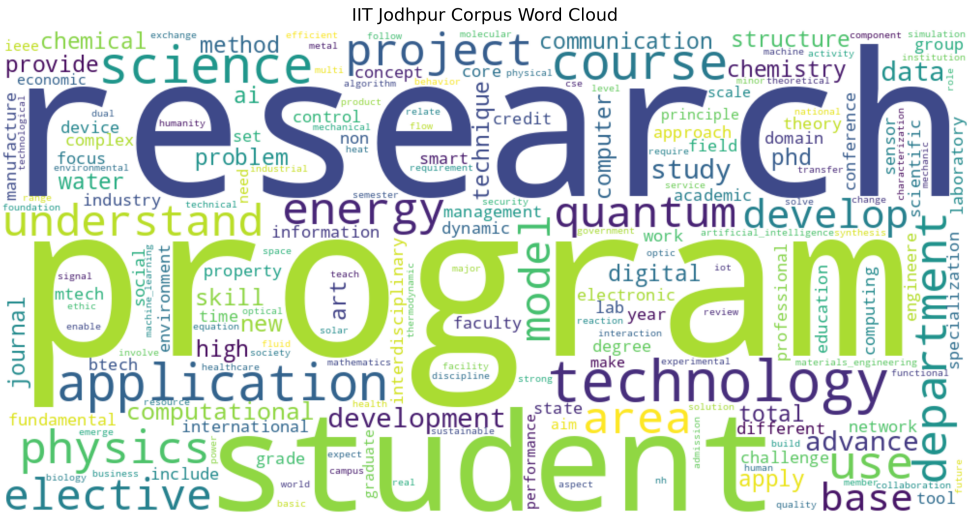


Figure 1: Word cloud of the cleaned IIT Jodhpur corpus

### 1.2 Task 2: Model Training

I trained two Word2Vec variants from scratch:

- **CBOW**: predicts the center word from surrounding context words.
- **Skip-gram with Negative Sampling**: predicts surrounding context words from the center word.

The hyperparameter study varied:

- embedding dimension: {100, 200, 300}
- context window size: {2, 5}
- number of negative samples: {5, 10}

Common training settings were: learning rate = 0.005, batch size = 128, maximum epochs = 120, early stopping based on validation loss, and a fixed validation split for fair comparison.

Among the saved runs, the best CBOW configuration was `dim=100, win=5, neg=5` with best validation loss **1.9062**. The best Skip-gram configuration was `dim=100, win=2, neg=5` with best validation loss **2.3729**. Overall, CBOW performed better than Skip-gram on this corpus under the selected validation criterion.

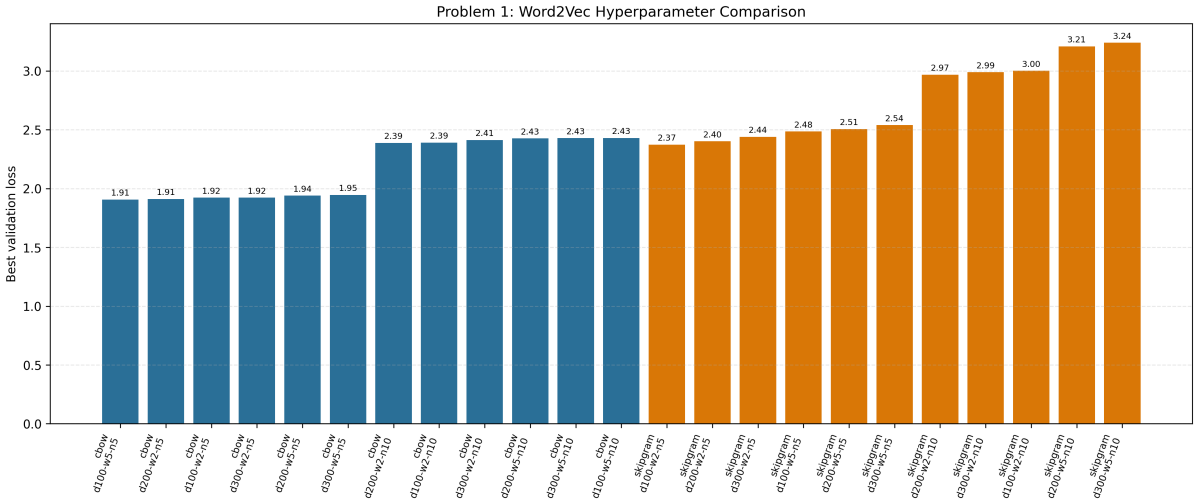


Figure 2: Validation-loss comparison across Word2Vec hyperparameter settings

Table 2: Problem 1 experiment results

| Model | Dim | Win | Neg | Best Val | Final Loss | Best Epoch | Epochs Run |
|---|---|---|---|---|---|---|---|
| CBOW | 100 | 2 | 5 | 1.9220 | 0.5214 | 8 | 40 |
| CBOW | 100 | 2 | 10 | 2.3907 | 0.7165 | 10 | 42 |
| CBOW | 100 | 5 | 5 | 1.9062 | 0.7064 | 10 | 42 |
| CBOW | 100 | 5 | 10 | 2.4293 | 0.9034 | 15 | 47 |
| CBOW | 200 | 2 | 5 | 1.9110 | 0.3970 | 6 | 38 |
| CBOW | 200 | 2 | 10 | 2.3884 | 0.5554 | 8 | 40 |
| CBOW | 200 | 5 | 5 | 1.9393 | 0.5866 | 7 | 39 |
| CBOW | 200 | 5 | 10 | 2.4263 | 0.8020 | 9 | 41 |
| CBOW | 300 | 2 | 5 | 1.9221 | 0.3716 | 4 | 36 |
| CBOW | 300 | 2 | 10 | 2.4122 | 0.4892 | 7 | 39 |
| CBOW | 300 | 5 | 5 | 1.9459 | 0.5223 | 6 | 38 |
| CBOW | 300 | 5 | 10 | 2.4285 | 0.7107 | 8 | 40 |
| SKIPGRAM | 100 | 2 | 5 | 2.3729 | 1.0286 | 6 | 38 |
| SKIPGRAM | 100 | 2 | 10 | 3.0013 | 1.4136 | 8 | 40 |
| SKIPGRAM | 100 | 5 | 5 | 2.4848 | 1.3981 | 6 | 38 |
| SKIPGRAM | 200 | 2 | 5 | 2.4010 | 0.8610 | 5 | 37 |
| SKIPGRAM | 200 | 2 | 10 | 2.9679 | 1.2221 | 6 | 38 |
| SKIPGRAM | 200 | 5 | 5 | 2.5059 | 1.2317 | 5 | 37 |
| SKIPGRAM | 200 | 5 | 10 | 3.2084 | 1.6302 | 7 | 15 |
| SKIPGRAM | 300 | 2 | 5 | 2.4402 | 0.8185 | 4 | 36 |
| SKIPGRAM | 300 | 2 | 10 | 2.9889 | 1.1429 | 6 | 38 |
| SKIPGRAM | 300 | 5 | 5 | 2.5395 | 1.1759 | 4 | 36 |
| SKIPGRAM | 300 | 5 | 10 | 3.2395 | 1.5339 | 7 | 15 |

### 1.3 Task 3: Semantic Analysis

For semantic analysis, I used cosine similarity on the best CBOW and best Skip-gram models. The assignment listed `exam` twice, so I kept `exam` once and used `hostel` as the fifth query word to make the analysis less repetitive.

### 1.3.1 Nearest Neighbors

Table 3: Top-5 nearest neighbors from the best CBOW model

| Word | Top-5 neighbors |
|---|---|
| research | multidisciplinary (0.574), scholar (0.567), conduct (0.560), centre (0.558), excellence (0.551) |
| student | gymkhana (0.693), opportunity (0.593), complete (0.532), motivate (0.525), curricular (0.524) |
| phd | assistantship (0.733), fellowship (0.718), comprehensive_exam (0.658), scholarship (0.641), candidate (0.640) |
| exam | grade (0.629), average (0.610), semester (0.561), tuition (0.559), eligible (0.549) |
| hostel | room (0.771), court (0.742), dining (0.733), recreational (0.727), facility (0.697) |

Table 4: Top-5 nearest neighbors from the best Skip-gram model

| Word | Top-5 neighbors |
|---|---|
| research | assistant (0.613), encourage (0.561), scholar (0.551), excellence (0.550), multidisciplinary (0.540) |
| student | gymkhana (0.663), assistantship (0.609), internship (0.599), opportunity (0.579), requisite (0.574) |
| phd | assistantship (0.684), comprehensive_exam (0.632), program (0.631), scholarship (0.625), msc (0.613) |
| exam | grade (0.704), average (0.688), internship (0.669), thesis (0.645), semester (0.640) |
| hostel | dining (0.782), room (0.733), court (0.709), recreational (0.694), consist (0.677) |

These neighborhoods are semantically meaningful. For example:

- `hostel` clustered with `room`, `dining`, `recreational`, and `facility`, which is exactly what we expect from student-life pages.
- `phd` clustered with `assistantship`, `fellowship`, `scholarship`, and `comprehensive_exam`, reflecting academic regulations and doctoral program descriptions.
- `student` was close to `gymkhana`, `opportunity`, `internship`, and related campus terms, showing that the models captured student-life semantics.

### 1.3.2 Analogy Experiments

I selected four analogy experiments that gave the clearest and most meaningful outputs. These examples directly test academic-program relations, student-faculty relations, and PhD-specific academic structure.

Table 5: Analogy experiments on the best CBOW and Skip-gram models

| Example | Expected | CBOW top-3 | Skip-gram top-3 | Comment |
|---|---|---|---|---|
| UG : BTech :: PG : ? | MTech / MBA / MSc / MTech-PhD | mtech_phd, mtech, mba | mtech_phd, mtech, dual | Strong for both models |
| BTech : UG :: MTech : ? | postgraduate | postgraduate, msc, assistantship | postgraduate, phd, msc | Correct top-1 in both |
| Student : Gymkhana :: Faculty : ? | member / staff | member, administrative, position | member, administrative, expert | Good campus relation |
| Semester : Course :: PhD : ? | comprehensive_exam | comprehensive_exam, scholar, arise | comprehensive_exam, department, engineere | Correct top-1 in both |

The analogy results show that both models learned meaningful domain structure, but CBOW was slightly cleaner. The strongest examples are `UG : BTech :: PG : ?`, where both models return `mtech_phd` and `mtech`, `BTech : UG :: MTech : ?`, where both return `postgraduate` as the top answer, and `Semester : Course :: PhD : ?`, where both return `comprehensive_exam`. These are good results because they match real academic relationships present in IIT Jodhpur regulations and program descriptions.

### 1.4 Task 4: Visualization

To visualize the embedding space, I projected the best CBOW and best Skip-gram embeddings to 2D using PCA and t-SNE.

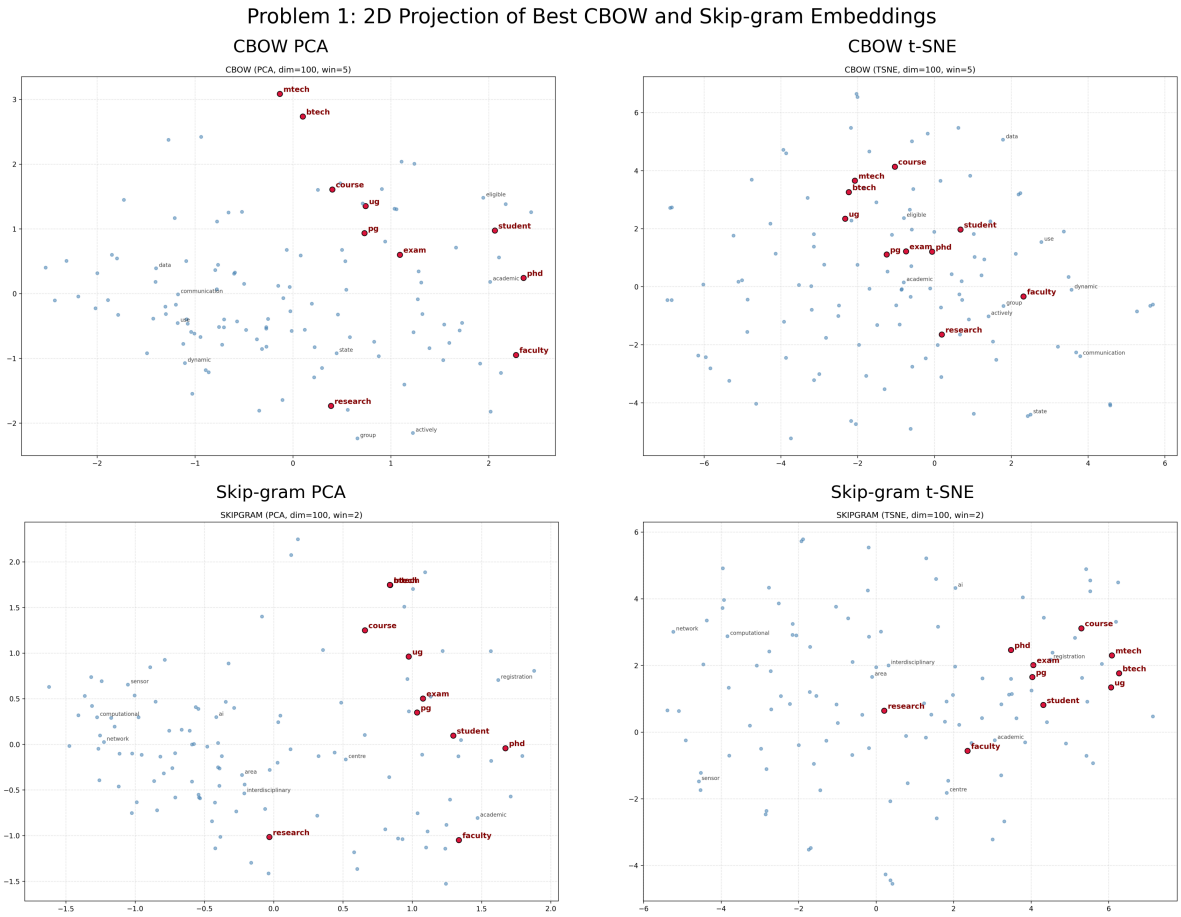


Figure 3: 2D visualization of the best CBOW and Skip-gram embeddings

The 2D projections show useful topical grouping:

- student-life words such as `hostel`, `room`, and `dining` form a visible local cluster,
- academic words such as `phd`, `exam`, `course`, and `faculty` stay relatively close,
- CBOW gives smoother and more compact local clusters,
- Skip-gram spreads points more aggressively, which is helpful for rarer words but also slightly noisier on this medium-sized corpus.

# 2 Problem 2: Character-Level Name Generation Using RNN Variants

## 2.1 Task 0: Dataset

The training dataset contains **1000 Indian names** stored in `q2/dataset/data.txt`. During loading, names were lowercased and internal whitespace was removed. One malformed line in the raw file contained internal tab characters, so that line was normalized before training. The resulting character vocabulary size is **28** including the special tokens `<pad>`, `<bos>`, and `<eos>`.

## 2.2 Task 1: Model Implementation

All three models were implemented from scratch as **character-level next-character generators**. Each training example consists of a prefix beginning with `<bos>` and the target is the next character or `<eos>`. At generation time, the model starts from `<bos>` and samples one character at a time until `<eos>` is produced.

Shared hyperparameters across all models:

- embedding size = 32
- hidden size = 64
- number of layers = 1
- learning rate = 0.003
- batch size = 64
- epochs = 20
- dropout = 0.15
- sampling temperature = 0.85
- generated names for evaluation = 300 per model

Table 6: Problem 2 model comparison

| Model | Params | Hidden | Layers | LR | Best Loss | Novelty (%) | Diversity |
|---|---|---|---|---|---|---|---|
| Vanilla RNN | 8988 | 64 | 1 | 0.003 | 1.7044 | 88.67 | 0.963 |
| BLSTM | 54684 | 64 | 1 | 0.003 | 1.3852 | 58.67 | 0.900 |
| RNN + Attention | 19165 | 64 | 1 | 0.003 | 1.6474 | 86.00 | 0.970 |

Architectures used:

- **Vanilla RNN**: character embeddings → unidirectional RNN → final hidden state → linear output layer.
- **BLSTM**: character embeddings → bidirectional LSTM → concatenated forward and backward final states → linear output layer.
- **RNN + Attention**: character embeddings → unidirectional RNN → additive attention over all prefix states → concatenated context and final state → linear output layer.

### 2.3 Task 2: Quantitative Evaluation

I used the following metrics:

- **Novelty Rate**: percentage of generated names not present in the training set.
- **Diversity**: number of unique generated names divided by the total number of generated names.

The best novelty came from the **Vanilla RNN** at **88.67%**, while the best diversity came from **RNN + Attention** at **0.970**. The BLSTM achieved the lowest training loss but also copied many more training names, which reduced both novelty and diversity.

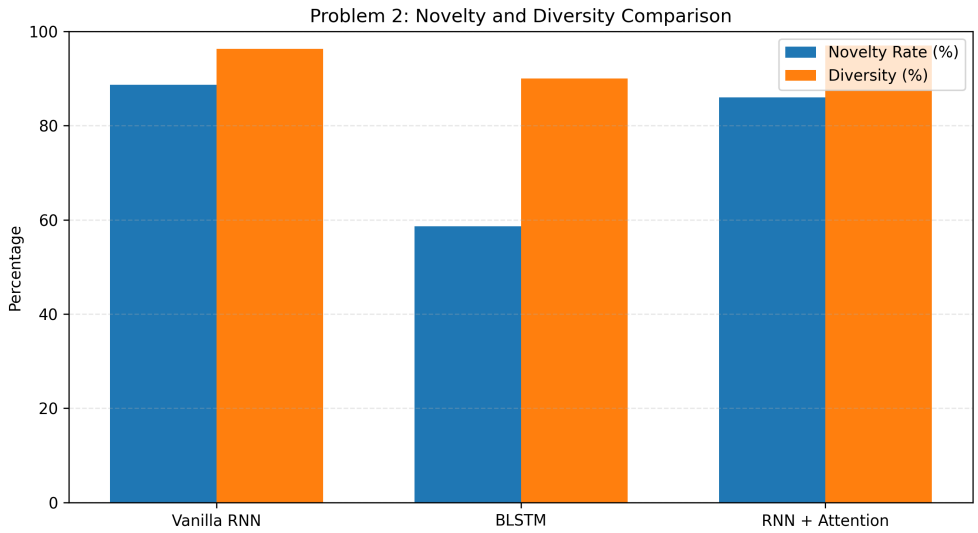


Figure 4: Problem 2 novelty and diversity comparison

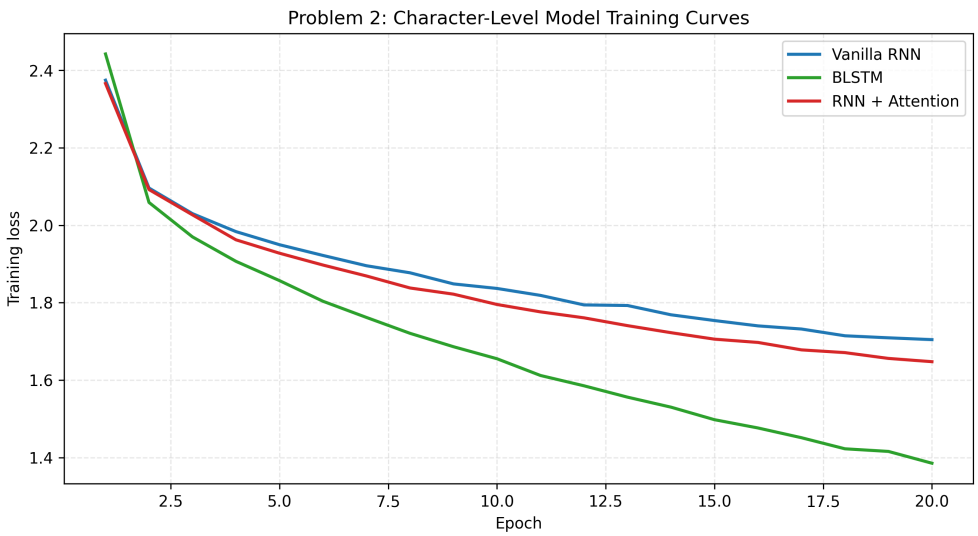


Figure 5: Training-loss curves for the three character-level models

### 2.4 Task 3: Qualitative Analysis

Table 7: Qualitative comparison of generated names

| Model | Realism | Common failure modes | Representative samples |
|---|---|---|---|
| Vanilla RNN | Produces recognizable syllable patterns, but it is the most likely to stop early or copy common training names. | memorization of training names (34/300); duplicate sampling (11 repeats); premature termination producing very short names (1); over-extended names or stitched fragments (1) | shasam, sankas, vitesh, radhsura, saheet, raghivan, lakhila, ramakam |
| BLSTM | Usually produces the smoothest global name structure, with stronger endings and more balanced syllable transitions. | memorization of training names (124/300); duplicate sampling (30 repeats); premature termination producing very short names (2) | manjo, ramak, prabhavti, amabi, ranju, chandrakant, jayakesh, yusman |
| RNN + Attention | Often captures useful prefix cues and generates varied names, though it occasionally stitches together fragments from different patterns. | memorization of training names (42/300); duplicate sampling (9 repeats); premature termination producing very short names (3); over-extended names or stitched fragments (1); character repetition artifacts (1) | dipender, jatil, sangeeta, mukesh, anild, sutunta, buldev, balweeeet |

Main observations:

- **Vanilla RNN** produced the highest novelty and many plausible names, but it sometimes stopped too early or stitched endings together.
- **BLSTM** produced the smoothest phonetic flow and the most realistic surface forms, but it showed the strongest memorization of training names.
- **RNN + Attention** gave the best balance between novelty and diversity, and attention helped preserve useful prefix information. Its main failures were occasional repeated letters and over-composed names such as `balweeeet`.

## 3 Conclusion

For Problem 1, CBOW produced cleaner semantic structure than Skip-gram on this IIT Jodhpur corpus, especially on nearest-neighbor quality and analogy consistency. For Problem 2, all three models generated plausible Indian-style names, but the comparison showed a clear tradeoff between fluency and originality: BLSTM was most fluent but least novel, Vanilla RNN achieved the highest novelty, and RNN + Attention gave the strongest overall balance.